

Universidad del Valle de Guatemala  
Facultad de Ingenieria  
Departamento de Ciencias de la Computacion  
Computacion Paralela y Distribuida - Seccion 20  
Ing. Sebastian Galindo, Ing. Juan Luis Garcia

# Proyecto #1

Screensaver de fluidos paralelizado con OpenMP  
Simulacion de Navier-Stokes (metodo Stable Fluids) con n-cuerpos

## Integrantes

TonitoMC  
Javier-Espana  
AngelEsquit

Guatemala, septiembre de 2026

# Índice

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>1. Introduccion</b>                                                | <b>2</b>  |
| <b>2. Antecedentes</b>                                                | <b>3</b>  |
| <b>3. Descripcion del problema</b>                                    | <b>4</b>  |
| <b>4. Metodologia: PCAM</b>                                           | <b>5</b>  |
| <b>5. Version secuencial</b>                                          | <b>6</b>  |
| <b>6. Paralelizacion iterativa</b>                                    | <b>7</b>  |
| 6.1. Paso 1: Barnes-Hut para n-body . . . . .                         | 7         |
| 6.2. Paso 2: paralelizar los loops independientes . . . . .           | 7         |
| 6.3. Paso 3: red-black Gauss-Seidel . . . . .                         | 7         |
| 6.4. Paso 4: ajuste de <code>schedule</code> . . . . .                | 7         |
| 6.5. Paso 5: ajuste de <code>collapse(2)</code> (cancelado) . . . . . | 8         |
| 6.6. Paso 6: flags de compilacion . . . . .                           | 8         |
| <b>7. Mecanismos de sincronizacion</b>                                | <b>9</b>  |
| <b>8. Resultados</b>                                                  | <b>10</b> |
| 8.1. Tabla maestra . . . . .                                          | 10        |
| 8.2. Escalabilidad por hilos . . . . .                                | 10        |
| 8.3. Barrido de <code>schedule/chunk</code> . . . . .                 | 11        |
| 8.4. Flags de build . . . . .                                         | 11        |
| <b>9. Conclusiones</b>                                                | <b>12</b> |
| <b>10.Recomendaciones</b>                                             | <b>13</b> |
| <b>A. Anexo 1: Diagrama de flujo del programa</b>                     | <b>15</b> |
| <b>B. Anexo 2: Catalogo de funciones</b>                              | <b>16</b> |
| <b>C. Anexo 3: Bitacora de pruebas</b>                                | <b>20</b> |

## 1. Introduccion

Este informe documenta el desarrollo de un screensaver de fluidos, paralelizado con OpenMP. El programa simula un fluido incompresible en 2D usando el metodo Stable Fluids de Jos Stam, con fuentes de tinta que se mueven por gravedad mutua (n-cuerpos).

El objetivo del proyecto es tomar una version secuencial funcional y mejorarla de forma iterativa, midiendo speedup y eficiencia en cada paso. Cada paso agrega una sola mejora (algoritmica o de paralelizacion) para que su impacto se pueda medir por separado.

## 2. Antecedentes

OpenMP es una API para programacion paralela de memoria compartida en C/C++/Fortran. Funciona mediante directivas de compilador (`#pragma omp`) que le indican al compilador que partes del codigo se pueden repartir entre varios hilos. A diferencia de MPI (memoria distribuida), todos los hilos de OpenMP comparten el mismo espacio de memoria, lo cual simplifica la programacion pero exige cuidado con condiciones de carrera.

El problema elegido, simulacion de fluidos, es un caso clasico de paralelizacion de memoria compartida: la malla se puede dividir en regiones, y (con el cuidado correcto de dependencias) cada region se puede actualizar en un hilo distinto. El metodo de Stam (*Stable Fluids*) resuelve las ecuaciones de Navier-Stokes con pasos incondicionalmente estables, lo que permite pasos de tiempo grandes sin que la simulacion explote numericamente [1].

Para el movimiento de las fuentes de tinta se implemento un sistema de n-cuerpos con gravedad mutua, acelerado con un quadtree Barnes-Hut [2], que reduce el costo de  $O(n^2)$  a  $O(n \log n)$ .

### 3. Descripcion del problema

El programa dibuja un lienzo de al menos 640x480 px donde varias fuentes de tinta de colores inyectan color y momentum en un campo de velocidad. El campo de velocidad se resuelve cada frame (difusion, adveccion, proyeccion de Hodge) y arrastra la tinta, generando remolinos. Las fuentes mismas se mueven por atraccion gravitacional mutua y rebotan en los bordes de la malla.

Parametros configurables por linea de comandos: resolucion de la malla (**-n**), cantidad de fuentes (**-f**), tamano de ventana (**-W/-H**), semilla aleatoria (**-s**), viscosidad (**-v**), difusion (**-d**) y pantalla completa (**-p**). Los valores por defecto y rangos validos estan en la Tabla 1.

| Flag      | Rango     | Default |
|-----------|-----------|---------|
| <b>-n</b> | 16 – 1024 | 256     |
| <b>-f</b> | 1 – 256   | 6       |
| <b>-W</b> | min. 640  | 1920    |
| <b>-H</b> | min. 480  | 1080    |
| <b>-v</b> | 0.0 – 1.0 | 0.0     |
| <b>-d</b> | 0.0 – 1.0 | 0.0001  |

Cuadro 1: Parametros de linea de comandos.

## 4. Metodologia: PCAM

Se aplico el metodo PCAM (Particionamiento, Comunicacion, Aglomeracion, Mapeo) para decidir que y como paralelizar:

**Particionamiento** La malla de la simulacion se piensa como un conjunto de celdas independientes. Cada operacion por celda (inyeccion de fuente, disipacion, adveccion, proyeccion, render) es, en principio, una tarea separada.

**Comunicacion** La mayoria de las operaciones solo leen el estado del frame anterior y escriben el nuevo, sin comunicacion entre celdas dentro del mismo paso. La excepcion es el solver de presion/difusion (Gauss-Seidel), que en su forma original lee valores ya actualizados de celdas vecinas en la misma iteracion, creando una dependencia estricta que impide paralelizar directamente.

**Agglomeracion** En vez de repartir celda por celda (overhead de sincronizacion altisimo), se agrupan filas en bloques (*chunks*) que cada hilo procesa de corrido.

**Mapeo** Los bloques se reparten entre los hilos disponibles via `#pragma omp parallel for`, con la politica de reparto (`schedule`) elegida segun el tamano de la malla (ver Seccion 6.4).

## 5. Version secuencial

La version secuencial (rama **sequential**) sirve de linea base fija para medir speedup. Usa Gauss-Seidel fila por fila para el solver y fuerza bruta  $O(f^2)$  para el n-body. No se modifica nunca despues de completada.

## 6. Paralelizacion iterativa

Siguiendo el espiritu de OpenMP como herramienta iterativa, la paralelizacion se hizo en pasos, cada uno en su propia rama, midiendo la misma bateria de pruebas despues de cada uno.

### 6.1. Paso 1: Barnes-Hut para n-body

Se reemplazo el calculo de gravedad de fuerza bruta ( $O(f^2)$ ) por un quadtree Barnes-Hut ( $O(f \log f)$ ). Este paso es puramente algoritmico, sin OpenMP todavia. A las resoluciones de malla probadas, el solver de fluidos domina tanto el costo que el cambio no se nota en el FPS total (el n-body es una fraccion pequena del trabajo por frame).

### 6.2. Paso 2: paralelizar los loops independientes

Se agrego `#pragma omp parallel for` a los sitios donde cada celda de salida es independiente: inyeccion de fuentes, disipacion de tinta, adveccion, proyeccion de Hodge y render. El solver de presion/difusion seguia secuencial. Con 20 hilos, este paso ya da un salto importante de FPS (Tabla 2), pero el techo sigue siendo el solver.

### 6.3. Paso 3: red-black Gauss-Seidel

El cambio mas grande de toda la cadena. El solver de Gauss-Seidel original lee y escribe la misma malla en el mismo barrido, celda por celda en orden, lo que crea una dependencia secuencial estricta. La solucion es un recorrido *red-black*: se divide la malla en un tablero de ajedrez por paridad  $(i + j) \bmod 2$ . Los 4 vecinos de una celda de un color son siempre del color opuesto, entonces todas las celdas del mismo color se pueden actualizar en paralelo sin pisarse entre si. Cada iteracion pasa a ser: actualizar todas las rojas (barrera implicita), actualizar todas las negras (barrera implicita).

Este cambio finalmente paraleliza el hotspot principal del programa. El resultado es el salto de rendimiento mas grande del proyecto: a la configuracion default ( $n = 256$ ,  $f = 6$ ), el FPS pasa de 25.20 (Paso 2) a 88.27 (Paso 3), un 3,5× adicional.

### 6.4. Paso 4: ajuste de schedule

Con el hotspot ya paralelo, se probo si el `schedule(dynamic, 16)` usado en todo el codigo era la mejor eleccion. Se cambiaron temporalmente los `#pragma` a `schedule(runtime)` (que se puede configurar sin recompilar via la variable de entorno `OMP_SCHEDULE`) y se barrio `static`, `dynamic` y `guided` con distintos tamanos de chunk, en malla chica y malla grande (Tabla 5).

El resultado principal: el `dynamic, 16` usado en todo el codigo es buena eleccion en malla chica, pero es la peor opcion razonable en malla grande. `static` (con el chunk por defecto del compilador) le saca 2,5× a `dynamic, 16` en  $n = 700$  (54.47 vs. 21.93 FPS).

Con este resultado, se aplico `schedule(static)` de verdad al codigo de la rama `04-schedule-tuning`, reemplazando el `schedule(dynamic, 16)` hardcodedo en todos los sitios del catalogo. Al remedir la bateria estandar con el cambio real (no solo con `OMP_SCHEDULE`), el resultado es un **tradeoff**, no una mejora universal: en malla chica `static` pierde un poco contra el `dynamic, 16` anterior (B6: 88.27 → 80.68 FPS, ~ -8,6 %), pero en malla grande gana fuerte (I6: 28.32 → 53.47 FPS, ~ +89 %; C6: 45.79 → 68.57 FPS, ~ +50 %). Ver Tabla 2 para los numeros completos.

### 6.5. Paso 5: ajuste de collapse(2) (cancelado)

Se planeo comparar `collapse(2)` contra paralelizar solo el loop externo, para ver el efecto en mallas chicas (menos filas que nucleos disponibles). Este paso se cancelo antes de obtener numeros confiables y no se incluye en los resultados de este informe.

### 6.6. Paso 6: flags de compilacion

Se probo cambiar el flag de optimizacion del compilador de `-O2` (usado hasta ahora) a `-O3`, y adicionalmente `-march=native` para autovectorizacion especifica del CPU de la maquina de pruebas. Los resultados estan en la Tabla 6.

`-O3` da una ganancia real de  $\sim 11\%$  en malla grande sobre `-O2`, sin costo en malla chica. `-march=native` no aporta nada adicional sobre `-O3` solo, lo que sugiere que los loops calientes no se benefician de registros vectoriales mas anchos en este CPU, o que el set de instrucciones base que asume GCC ya alcanza.

## 7. Mecanismos de sincronizacion

- **#pragma omp parallel for**: la forma principal de paralelismo en el proyecto. Reparte las iteraciones de un loop **for** entre los hilos disponibles, con una barrera implicita al final (todos los hilos esperan a que termine el ultimo antes de seguir).
- **#pragma omp parallel + #pragma omp for**: usado en el solver (Seccion 6.3) para abrir una sola region paralela que envuelve las 20 iteraciones del Gauss-Seidel red-black, en vez de abrir/cerrar el team de hilos 40 veces por frame (costo fijo de creacion de hilos evitado).
- **#pragma omp single**: dentro de la region paralela del solver, aplica la condicion de frontera con un solo hilo mientras el resto espera en la barrera implicita del **single**, evitando que varios hilos escriban la misma celda de borde a la vez.
- **schedule(dynamic, chunk)**: politica de reparto de trabajo por defecto usada en todo el codigo. Reparte bloques de **chunk** iteraciones bajo demanda: un hilo que termina su bloque pide el siguiente en vez de quedarse ocioso esperando a los demas. La Seccion 6.4 muestra que no siempre es la mejor opcion.
- **collapse(2)**: en los loops anidados de adveccion, proyeccion y render, funde las dos dimensiones (fila, columna) en un solo rango repartible. Sin esto, una malla mas angosta que la cantidad de hilos dejaria nucleos sin trabajo.
- **Variables privadas (private(...))**: cada hilo necesita su propia copia de las variables escalares temporales del cuerpo del loop (por ejemplo, coordenadas de origen interpoladas en adveccion). Sin esto, los hilos pisarian el mismo valor compartido y corromperian el resultado.

No se usaron mutex ni secciones criticas explicitas: el diseno evita condiciones de carrera por construccion (cada celda de salida se escribe una sola vez, por un solo hilo, en cada paso), asi que las barreras implicitas de **omp for** y el **single** para la frontera son suficientes.

## 8. Resultados

### 8.1. Tabla maestra

La Tabla 2 muestra el FPS promedio por configuracion (malla  $\times$  fuentes), en cada paso de la cadena, con 20 hilos donde aplica. Metodologia: cada fila se corrio con semilla fija (`-s 42`), se promedia el FPS ignorando las primeras lineas de calentamiento.

| Fila               | sequential | 01-rb-tree | 02-omp-loops | 03-omp-solver<br>(dynamic,16) | 04-schedule<br>(static) |
|--------------------|------------|------------|--------------|-------------------------------|-------------------------|
| A6 (n=64, f=6)     | 17.21      | 17.19      | 118.40       | 88.04                         | 83.87                   |
| A12 (n=64, f=12)   | 17.16      | 17.20      | 117.43       | 88.31                         | 84.39                   |
| B6 (n=256, f=6)    | 10.52      | 10.51      | 25.20        | 88.27                         | 80.68                   |
| B12 (n=256, f=12)  | 10.95      | 10.95      | 25.24        | 88.66                         | 80.92                   |
| C6 (n=512, f=6)    | 3.75       | 3.76       | 6.72         | 45.79                         | 68.57                   |
| C12 (n=512, f=12)  | 4.30       | 4.30       | 6.89         | 52.33                         | 69.53                   |
| H6 (n=600, f=6)    | 2.69       | 2.69       | 4.46         | 33.72                         | 61.41                   |
| H12 (n=600, f=12)  | 3.09       | 3.09       | 4.79         | 41.39                         | 62.29                   |
| I6 (n=700, f=6)    | 1.90       | 1.91       | 3.00         | 28.32                         | 53.47                   |
| I12 (n=700, f=12)  | 2.10       | 2.10       | 3.21         | 35.57                         | 54.76                   |
| D6 (n=1024, f=6)   | 0.92       | 0.92       | 1.15         | 5.10                          | 4.64                    |
| D12 (n=1024, f=12) | 1.00       | 0.99       | 1.23         | 6.32                          | 3.21                    |
| E (n=256, f=4)     | 7.52       | 7.52       | 24.39        | 87.09                         | n/m                     |
| F (n=256, f=64)    | 11.39      | 11.39      | 25.36        | 87.28                         | n/m                     |
| G (n=256, f=256)   | 11.37      | 11.38      | 25.30        | 85.07                         | n/m                     |

Cuadro 2: FPS promedio por paso de la cadena de paralelizacion. n/m = no medido con el schedule nuevo todavia.

En la configuracion default ( $n = 256$ ,  $f = 6$ ), el FPS pasa de 10.52 (secuencial) a 88.27 (Paso 3), un speedup acumulado de  $\sim 8,4\times$ . El Paso 4 (`static`) es un tradeoff frente a Paso 3: pierde un poco en malla chica pero gana fuerte en malla grande (hasta  $\sim +89\%$  en  $n=700$ , Tabla 2).

### 8.2. Escalabilidad por hilos

La Tabla 3 muestra el FPS variando `OMP_NUM_THREADS` en la fila B ( $n=256$ ,  $f=6$ ). Antes del Paso 3 (solver todavia secuencial), el escalado se estanca alrededor de  $2,6\times$  (techo de Amdahl: el solver secuencial limita cuanto puede acelerar el resto). Desde el Paso 3 en adelante, con el solver ya paralelo, el escalado sube a  $\sim 6\times$ .

| Hilos | FPS 02-omp-loops | FPS 03-omp-solver | FPS 04-schedule |
|-------|------------------|-------------------|-----------------|
| 1     | 9.95             | 14.72             | 14.72           |
| 2     | 14.94            | 25.39             | 25.52           |
| 4     | 20.33            | 45.94             | 45.91           |
| 8     | 23.57            | 67.84             | 68.27           |
| 16    | 25.52            | 83.27             | 83.66           |
| 20    | 25.57            | 87.76             | 87.72           |

Cuadro 3: FPS vs. cantidad de hilos, fila B ( $n=256$ ,  $f=6$ ).

Tambien se corrio la misma curva en una malla grande (n=700, f=12), comparando el `schedule` actual contra `static` (el mejor encontrado para malla grande, Tabla 4). Con `static`, 20 hilos da 48.11 FPS contra los 34.59 FPS del `dynamic`, 16 actual en la misma fila: +39 % solo cambiando el `schedule`.

| Hilos | FPS <code>static</code> | FPS <code>dynamic</code> , 16 (actual) |
|-------|-------------------------|----------------------------------------|
| 1     | 6.82                    | —                                      |
| 2     | 15.25                   | —                                      |
| 4     | 30.58                   | —                                      |
| 8     | 41.02                   | —                                      |
| 16    | 53.80                   | —                                      |
| 20    | 48.11                   | 34.59                                  |

Cuadro 4: FPS vs. hilos en malla grande (n=700, f=12).

Con `static` el pico esta en 16 hilos, no en 20: a 700 filas, el reparto estatico en 20 partes no cae tan parejo como en 16.

### 8.3. Barrido de `schedule/chunk`

| OMP_SCHEDULE                       | FPS n=256    | FPS n=700    |
|------------------------------------|--------------|--------------|
| <code>static</code> (default)      | 83.40        | <b>54.47</b> |
| <code>static</code> , 4            | 83.57        | 43.01        |
| <code>static</code> , 16           | 84.53        | 46.87        |
| <code>static</code> , 64           | 72.89        | 44.42        |
| <code>dynamic</code> , 1           | 31.19        | 5.55         |
| <code>dynamic</code> , 4           | 64.36        | 15.77        |
| <code>dynamic</code> , 16 (actual) | 87.58        | 21.93        |
| <code>dynamic</code> , 64          | 76.88        | 2.02         |
| <code>guided</code> (default)      | <b>96.80</b> | 49.42        |
| <code>guided</code> , 16           | 94.71        | 3.43         |

Cuadro 5: Barrido de `schedule/chunk`, 20 hilos, f=6.

### 8.4. Flags de build

| Build             | FPS n=256, f=6 | FPS n=700, f=12 |
|-------------------|----------------|-----------------|
| -O2 (actual)      | 83.74          | 32.18           |
| -O3               | 83.16          | 35.72           |
| -O3 -march=native | 84.03          | 35.76           |

Cuadro 6: FPS por flag de optimizacion, 20 hilos.

## 9. Conclusiones

- Paralelizar el solver de presion/difusion (red-black Gauss-Seidel) fue, por lejos, el cambio de mayor impacto: FPS default subio de 25.20 a 88.27 (3,5×), y la escalabilidad por hilos paso de  $\sim 2,6\times$  a  $\sim 6\times$ .
- Antes de paralelizar el hotspot real, paralelizar el resto del programa (Paso 2) ayuda, pero el techo de Amdahl limita cuanto se puede ganar mientras el cuello de botella siga secuencial.
- La eleccion de `schedule` de OpenMP importa mas de lo esperado y no tiene un ganador universal: `dynamic`, 16 era mejor en malla chica, `static` gana fuerte en malla grande (2,5× en el barrido, hasta +89% al aplicarlo de verdad al codigo). Se aplico `static` a la rama `04-schedule-tuning`; el resultado real confirma el tradeoff: pierde  $\sim 8\%$  en malla chica para ganar hasta  $\sim 89\%$  en malla grande.
- Cambiar `-O2` a `-O3` da una ganancia gratuita de  $\sim 11\%$  en malla grande sin tocar codigo. `-march=native` no aporta nada extra sobre `-O3` en este CPU.
- Barnes-Hut para el n-body no se nota en el FPS a las resoluciones probadas porque el solver de fluidos domina el costo total; el algoritmo de n-body es una fraccion pequena del trabajo por frame.

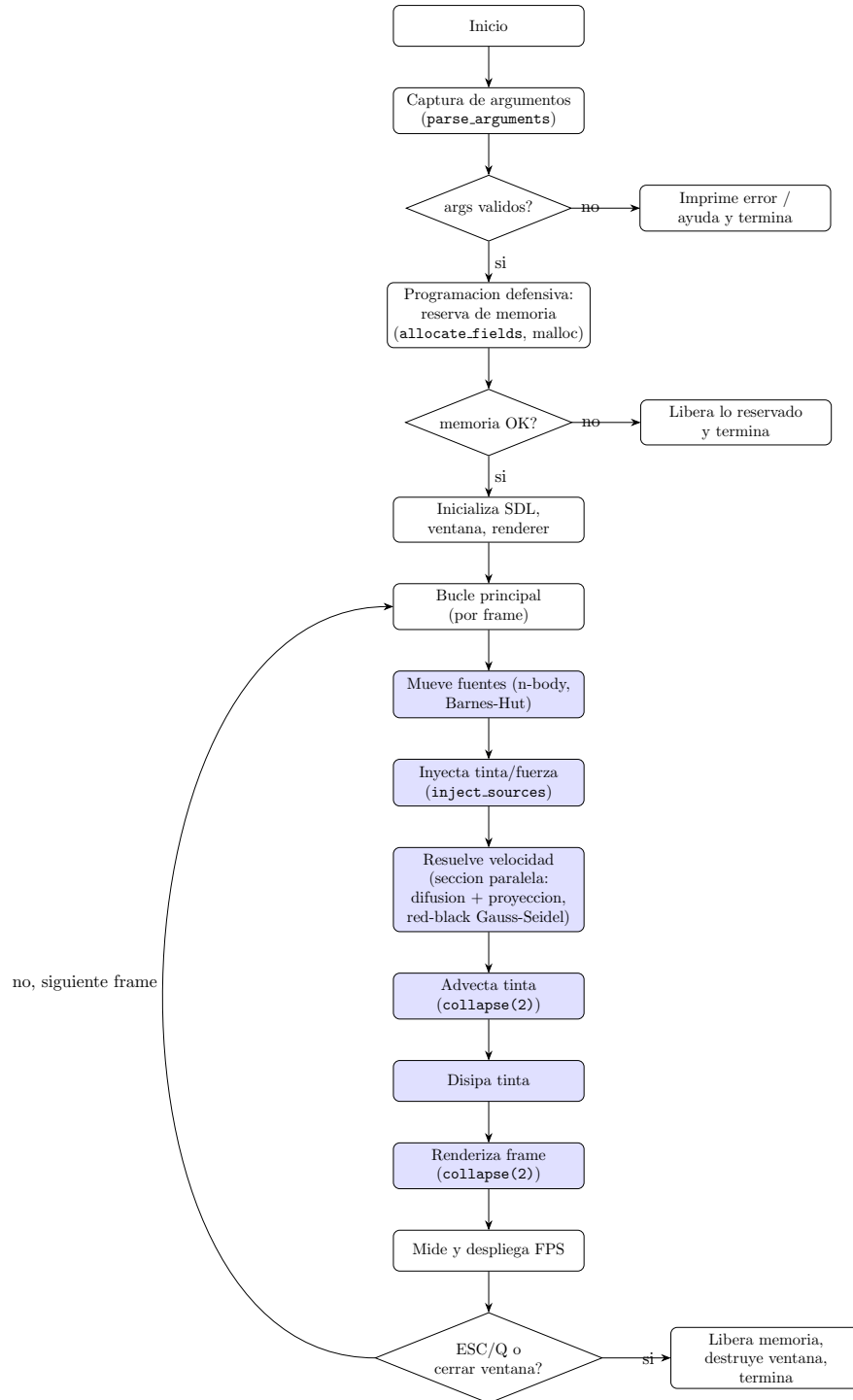
## 10. Recomendaciones

- Ya se aplico `static` en `04-schedule-tuning`. Si el uso real del programa favorece mallas chicas ( $n \leq 256$ ), vale la pena reconsiderar, ya que ahí `dynamic`, `16` sigue siendo mejor; para mallas grandes, `static` es la mejor eleccion medida. Una opcion futura es dejarlo configurable via `schedule(runtime)` y elegir en base a `-n` al arrancar.
- Cambiar `-02` a `-03` en el `Makefile`: es una ganancia gratuita sin riesgo.
- Retomar el Paso 5 (`collapse(2)`) con una metodologia mas controlada (por ejemplo, sin renderizado en pantalla completa) si se quiere ese numero para mallas chicas con pocos nucleos.
- Para mallas mayores a  $n = 1024$  o cantidades de fuentes mayores a 256, medir de nuevo: el comportamiento del schedule optimo podria cambiar fuera del rango probado.

## Referencias

- [1] Stam, J. (1999). *Stable Fluids*. En Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99), 121-128.
- [2] Barnes, J., & Hut, P. (1986). *A hierarchical  $O(N \log N)$  force-calculation algorithm*. Nature, 324(6096), 446-449.
- [3] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. <https://www.openmp.org/specifications/>
- [4] Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

## A. Anexo 1: Diagrama de flujo del programa



Los bloques en azul son las secciones paralelizadas con OpenMP. La sincronizacion entre ellas es implicita: cada `#pragma omp for` tiene una barrera al final, y la seccion de velocidad usa `#pragma omp single` para la condicion de frontera dentro de su propia region paralela (ver Seccion 6.3).

## B. Anexo 2: Catalogo de funciones

### config.c

| Funcion                      | Entradas                                            | Salidas                                                                              | Descripcion                                                                      |
|------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>parse_arguments</code> | <code>argc</code> (int), <code>argv</code> (char**) | <code>config</code> (Config*, por referencia), retorna int (1 ok, 0 error, -1 ayuda) | Recorre <code>argv</code> y llena <code>config</code> con los valores validados. |
| <code>read_int</code>        | <code>text</code> (char*), rango min/-max           | <code>out</code> (int*), retorna 1/0                                                 | Convierte texto a int validando formato y rango.                                 |
| <code>read_float</code>      | <code>text</code> (char*), rango min/-max           | <code>out</code> (float*), retorna 1/0                                               | Igual que <code>read_int</code> pero para float.                                 |
| <code>print_help</code>      | <code>program_name</code> (char*)                   | ninguna (imprime a stdout)                                                           | Imprime la pantalla de uso del programa.                                         |

### fields.c

| Funcion                      | Entradas                                                          | Salidas                     | Descripcion                                                                                   |
|------------------------------|-------------------------------------------------------------------|-----------------------------|-----------------------------------------------------------------------------------------------|
| <code>allocate_fields</code> | <code>fields</code> (FluidFields*), <code>resolution</code> (int) | retorna int (1 ok, 0 error) | Reserva memoria para los 12 campos de la simulacion (velocidad, tinta, presion, divergencia). |
| <code>free_fields</code>     | <code>fields</code> (FluidFields*)                                | ninguna                     | Libera los 12 campos y pone los punteros en NULL.                                             |
| <code>clear_fields</code>    | <code>fields</code> (FluidFields*)                                | ninguna                     | Pone todos los campos en cero (usado al reiniciar con la tecla R).                            |

### sources.c

| Funcion                   | Entradas                                                                              | Salidas | Descripcion                                                          |
|---------------------------|---------------------------------------------------------------------------------------|---------|----------------------------------------------------------------------|
| <code>init_sources</code> | <code>sources</code> (InkSource*), <code>count</code> , <code>resolution</code> (int) | ninguna | Inicializa posicion, velocidad, masa, color y fuerza de cada fuente. |

|                             |                                                                                                                                                                                                |                                         |                                                                                                 |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>inject_sources</code> | <code>sources</code> ( <code>InkSource*</code> ),<br><code>count</code> ( <code>int</code> ), <code>fields</code> ( <code>FluidFields*</code> ),<br><code>aspect</code> ( <code>float</code> ) | ninguna (modifica <code>fields</code> ) | Deposita tinta y momentum de cada fuente en un vecindario gaussiano.                            |
| <code>dissipate_ink</code>  | <code>fields</code> ( <code>FluidFields*</code> )                                                                                                                                              | ninguna                                 | Reduce la tinta de cada celda por un factor fijo (paralelo con <code>omp parallel for</code> ). |

`solver.c`

| Funcion                     | Entradas                                                                                                                                                                                                                            | Salidas                        | Descripcion                                                                                                        |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>apply_boundary</code> | <code>bnd_type</code> ( <code>int</code> ), <code>field</code> ( <code>float*</code> )                                                                                                                                              | ninguna                        | Aplica condiciones de frontera (rebote en paredes).                                                                |
| <code>add_source</code>     | <code>dest</code> , <code>source</code> ( <code>float*</code> ), <code>dt</code> ( <code>float</code> ), <code>n</code> ( <code>int</code> )                                                                                        | ninguna                        | Suma el termino fuente a cada celda (paralelo).                                                                    |
| <code>relax_color</code>    | <code>field</code> , <code>field_prev</code> ( <code>float*</code> ), <code>a</code> , <code>inv_c</code> ( <code>float</code> ), <code>parity</code> ( <code>int</code> )                                                          | ninguna                        | Actualiza las celdas de una paridad del tablero red-black (paralelo, dentro de region <code>omp parallel</code> ). |
| <code>solve_linear</code>   | <code>bnd_type</code> ( <code>int</code> ), <code>field</code> , <code>field_prev</code> ( <code>float*</code> ), <code>a</code> , <code>c</code> ( <code>float</code> )                                                            | ninguna                        | Corre las 20 iteraciones de Gauss-Seidel red-black dentro de una sola region paralela.                             |
| <code>diffuse</code>        | <code>bnd_type</code> ( <code>int</code> ), <code>field</code> , <code>field_prev</code> ( <code>float*</code> ), <code>diff</code> , <code>dt</code> ( <code>float</code> ), <code>n</code> ( <code>int</code> )                   | ninguna                        | Arma los coeficientes y llama a <code>solve_linear</code> para difusion implicita.                                 |
| <code>advect</code>         | <code>bnd_type</code> ( <code>int</code> ), <code>field</code> , <code>field_prev</code> ( <code>float*</code> ), <code>vel_x</code> , <code>vel_y</code> ( <code>float*</code> ), <code>dt</code> ( <code>float</code> )           | ninguna                        | Adveccion semi-Lagrangiana (paralelo, <code>collapse(2)</code> ).                                                  |
| <code>project</code>        | <code>vel_x</code> , <code>vel_y</code> , <code>pressure</code> , <code>divergence</code> ( <code>float*</code> )                                                                                                                   | ninguna                        | Proyeccion de Hodge: mide divergencia, resuelve presion, resta el gradiente de la velocidad.                       |
| <code>swap_fields</code>    | <code>field_a</code> , <code>field_b</code> ( <code>float**</code> )                                                                                                                                                                | ninguna (intercambia punteros) | Intercambia los punteros de dos buffers de campo.                                                                  |
| <code>ink_step</code>       | <code>ink</code> , <code>ink_prev</code> ( <code>float**</code> ), <code>vel_x</code> , <code>vel_y</code> ( <code>float*</code> ), <code>diff</code> , <code>dt</code> ( <code>float</code> ), <code>n</code> ( <code>int</code> ) | ninguna                        | Un paso completo (difusion + adveccion) para un canal de tinta.                                                    |

|                            |                                                                                         |         |                                                                                         |
|----------------------------|-----------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------|
| <code>velocity_step</code> | <code>fields</code> (FluidFields*),<br><code>viscosity</code> , <code>dt</code> (float) | ninguna | Un paso completo de velocidad: fuerza, difusion, proyeccion, autoadveccion, proyeccion. |
|----------------------------|-----------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------|

`nbody.c`

| Funcion                           | Entradas                                                                                                                        | Salidas                                        | Descripcion                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>bh_new_node</code>          | <code>cx</code> , <code>cy</code> , <code>half</code> (float)                                                                   | retorna int (indice del nodo)                  | Crea un nodo nuevo del quadtree Barnes-Hut.                                                   |
| <code>bh_quadrant</code>          | <code>n</code> (QuadNode*), <code>x</code> , <code>y</code> (float)                                                             | retorna int (0-3)                              | Determina en que cuadrante cae un punto.                                                      |
| <code>bh_subdivide</code>         | <code>idx</code> (int)                                                                                                          | ninguna                                        | Crea los 4 hijos de un nodo.                                                                  |
| <code>bh_insert</code>            | <code>idx</code> , <code>body</code> (int), <code>sources</code> (InkSource*), <code>depth</code> (int)                         | ninguna                                        | Inserta una fuente en el arbol recursivamente.                                                |
| <code>bh_accumulate</code>        | <code>idx</code> , <code>body</code> (int), <code>sources</code> (InkSource*), <code>acc_x</code> , <code>acc_y</code> (float*) | ninguna (acumula en <code>acc_x/acc_y</code> ) | Recorre el arbol acumulando aceleracion gravitacional sobre una fuente.                       |
| <code>update_nbody_sources</code> | <code>sources</code> (InkSource*), <code>count</code> , <code>resolution</code> (int)                                           | ninguna                                        | Reconstruye el arbol, calcula gravedad, integra posicion/velocidad y aplica rebote en bordes. |

`render.c`

| Funcion                      | Entradas                                                                                                                | Salidas       | Descripcion                                                                                            |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------|--------------------------------------------------------------------------------------------------------|
| <code>sample_bilinear</code> | <code>field</code> (float*), <code>fx</code> , <code>fy</code> (float)                                                  | retorna float | Interpola bilinealmente un campo en una posicion continua.                                             |
| <code>render_ink</code>      | <code>texture</code> (SDL_Texture*), <code>fields</code> (FluidFields*), <code>width</code> , <code>height</code> (int) | ninguna       | Vuelca la densidad de tinta a la textura SDL, un pixel a la vez (paralelo, <code>collapse(2)</code> ). |

`background.c`

| Funcion                      | Entradas                                                                                        | Salidas | Descripcion                                         |
|------------------------------|-------------------------------------------------------------------------------------------------|---------|-----------------------------------------------------|
| <code>init_background</code> | <code>stars</code> (Star*), <code>count</code> , <code>width</code> , <code>height</code> (int) | ninguna | Coloca estrellas de fondo en posiciones aleatorias. |

|                                |                                                                                               |         |                                      |
|--------------------------------|-----------------------------------------------------------------------------------------------|---------|--------------------------------------|
| <code>update_background</code> | <code>stars (Star*)</code> , <code>count (int)</code>                                         | ninguna | Avanza el parpadeo de cada estrella. |
| <code>draw_background</code>   | <code>renderer (SDL_Renderer*)</code> , <code>stars (Star*)</code> , <code>count (int)</code> | ninguna | Dibuja las estrellas de fondo.       |

**utils.c**

| Funcion                   | Entradas                             | Salidas       | Descripcion                                       |
|---------------------------|--------------------------------------|---------------|---------------------------------------------------|
| <code>random_range</code> | <code>min, max (float)</code>        | retorna float | Numero pseudoaleatorio uniforme en $[min, max]$ . |
| <code>clamp</code>        | <code>value, min, max (float)</code> | retorna float | Limita un valor al rango $[min, max]$ .           |

**main.c**

| Funcion           | Entradas                                             | Salidas                        | Descripcion                                                                                                    |
|-------------------|------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>main</code> | <code>argc (int)</code> , <code>argv (char**)</code> | retorna int (codigo de salida) | Punto de entrada: parsea argumentos, inicializa SDL y memoria, corre el bucle principal, libera todo al salir. |

## C. Anexo 3: Bitacora de pruebas

### Metodologia

Cada configuracion se corrio con semilla fija (`-s 42`), ignorando las primeras lineas de FPS (calentamiento/cache fria), y promediando el resto. La bateria estandar (15 configuraciones, Tabla 16) se repitio identica despues de cada paso de la cadena; son 15 mediciones por paso, mas de las 10 minimas requeridas.

| Fila   | -n   | -f   | Motivo                             |
|--------|------|------|------------------------------------|
| A6/A12 | 64   | 6/12 | mallla chica, referencia rapida    |
| B6/B12 | 256  | 6/12 | configuracion default              |
| C6/C12 | 512  | 6/12 | mallla grande, domina el solver    |
| H6/H12 | 600  | 6/12 | rango de FPS medio observado       |
| I6/I12 | 700  | 6/12 | extremo superior de ese rango      |
| D6/D12 | 1024 | 6/12 | mallla muy grande, limite superior |
| E      | 256  | 4    | pocas fuentes                      |
| F      | 256  | 64   | muchas fuentes                     |
| G      | 256  | 256  | limite superior de fuentes         |

Cuadro 16: Bateria estandar de pruebas (15 configuraciones).

Para el barrido de `schedule` (Tabla 5) se probaron 10 combinaciones de politica/chunk, cada una en 2 tamanos de mallla: 20 mediciones. Para la escalabilidad por hilos (Tablas 3 y 4) se probaron 6 cantidades de hilos por curva.

### Captura de una corrida

A continuacion, salida real de consola de una corrida de referencia ( $n = 256$ ,  $f = 6$ , semilla 42), mostrando el arranque, calentamiento y estabilizacion del FPS:

```
Fluid simulation (Navier-Stokes) - SEQUENTIAL version
Grid       : 256 x 256 cells (65536 interior cells)
Sources    : 6
Window     : 1920 x 1080 px
Seed       : 42
visc / diff : 0.00000 / 0.00010
FPS= 12.73
FPS= 11.41
FPS= 12.96
FPS= 13.80
FPS= 75.53
FPS= 83.46
FPS= 83.82
FPS= 82.31
FPS= 84.26
FPS= 85.41
FPS= 84.09
FPS= 63.32
FPS= 74.58
FPS= 70.27
FPS= 77.99
```

Nota: la etiqueta "SEQUENTIAL version.<sup>en</sup> el encabezado es texto residual del archivo `main.c` (no se actualizo al paralelizar); el binario usado en esta corrida es la version paralela con OpenMP habilitado (20 hilos).

### Comando usado

```
1 timeout 30s ./ss -n <N> -f <F> -s 42 2>/dev/null \  
2 | grep "FPS=" | tail -n +6 \  
3 | awk -F'= ' '{s+=$2; c++} END {printf "FPS promedio: %.2f\n", s/c}'
```

El resto de las mediciones completas (las 15 filas de la bateria estandar por cada paso, el barrido de schedule y las curvas de escalabilidad) estan documentadas en `docs/Roadmap.md` dentro del repositorio del proyecto, junto con el historial de commits que muestra cuando se corrio cada bateria.